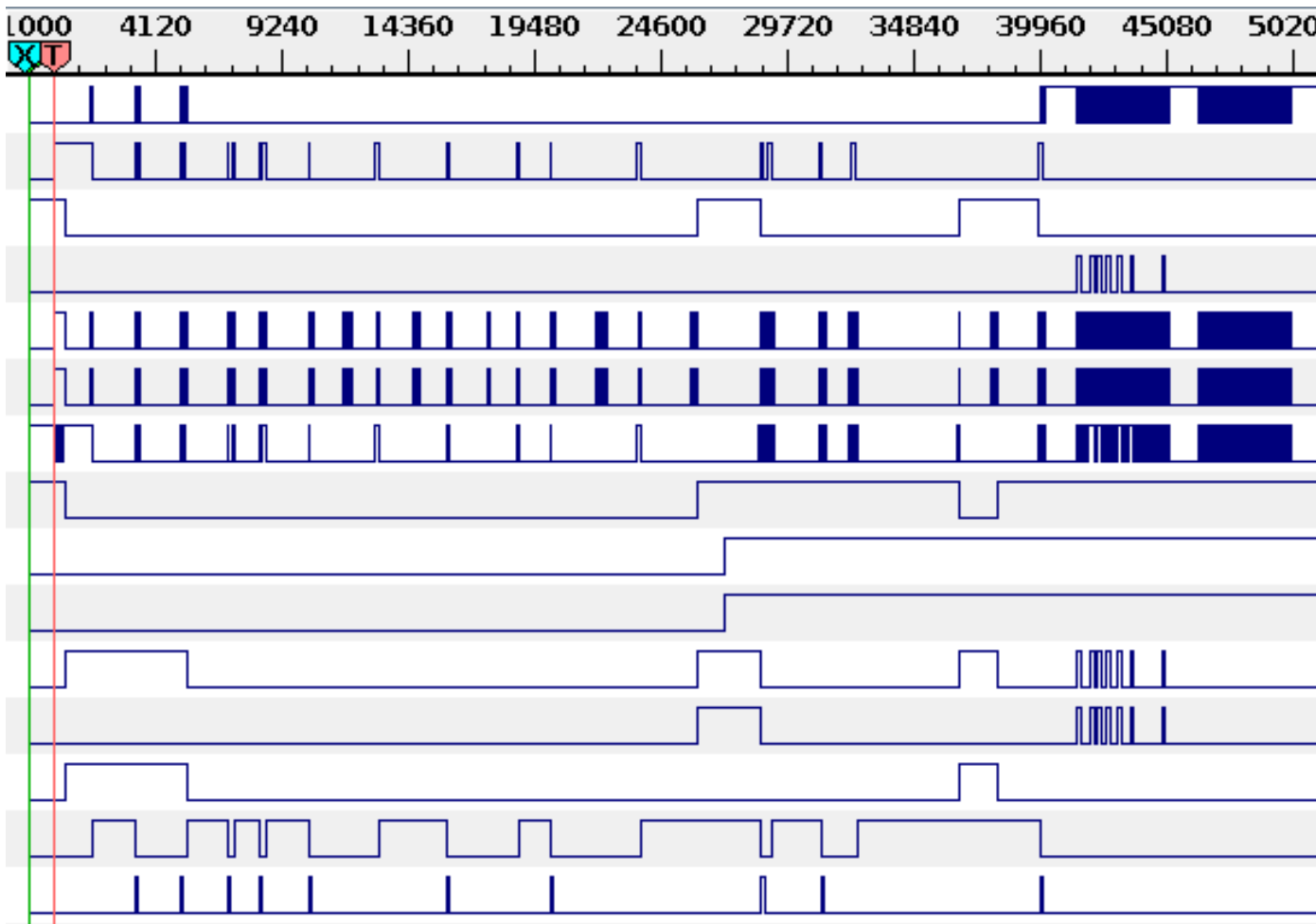


We understand the protocol 2-wire JTAG



2-wire JTAG (aka two-wire JTAG, aka CompactJTAG, aka cJTAG) is a newfangled interface that is part of the IEEE 1149.7-2009 standard. It provides the same and even greater functionality as a regular JTAG (IEEE 1149.1), but uses only two signals instead of four.

Unfortunately, neither the Russian nor the English segments of the Internet have any information about this standard, except for a few articles written by marketers. Nevertheless, some time ago, I had to deal with this standard on duty, and now you have a unique chance to get acquainted with the results of my research.

I assume that you are familiar with the usual JTAG. You can refresh your memory here: habrahabr.ru/post/199012

It is often believed that IEEE 1149.7 and 2-wire JTAG are synonymous. In fact, this is not the case, because IEEE 1149.7 includes modes that use both two and four signals. So 2-wire JTAG is a subset of IEEE 1149.7.

Officially, IEEE 1149.7 is called *IEEE Standard for Reduced-Pin and Enhanced-Functionality*. Despite the fact that, in addition to the two-conductor nature, IEEE 1149.7 offers still a lot of new features, in my experience it is the opportunity to save two legs on the microcircuit case that attracts developers the most.

Unfortunately, the creators of IEEE 1149.7 used hard drugs without any measure, so the monstrous document that firmly won first place in my personal hit parade of the most ugly written standards gave birth without any understatement. Besides the fact that it has grown almost ten times (1037 pages versus 139 for the old JTAG), it is also written in such a way that it is absolutely impossible to understand anything.

However, if you once face a chip with a chip that has instead of four JTAG legs (TCK, TMS, TDI and TDO) there are only two (TCKC and TMSC - "C" on the end means "Compact"), and the finished software to work with it will not, then you have to do something.

What an engineer usually does when he sees a microchip with an unidentified JTAG port in front of him. Of course, trying to figure out which devices are connected to it. You can find out by reading their IDCODE. For a regular JTAG, the procedure is simple and looks like described by the link www.fpga4fun.com/JTAG3.html:

1. First we determine the number of devices in the chain:
 - Go to Test Mode Reset
 - Go to Shift-IR
 - Via TDI, we push a bunch of units (how many do not mind, for example, 1000) in all IRs in the chain. Now all devices in BYPASS
 - Moving from Shift-IR to Shift-DR
 - Via TDI, we push in a bunch of zeros (how many do not mind, for example, 1000) to all DRs in the chain in order to reset them
 - We begin to push units through DRI through TDI. As soon as we get a unit from TDO, we stop. The number of devices in the chain is equal to the number of retracted units.
2. Then for each device we read IDCODE:
 - Go to Test Mode Reset - as a result, the address of the IDCODE register is recorded in all IRs.
 - Go to Shift-DR
 - We read from TDO 32 bits for each device (by the way, for this you have to push something into TDI). The first 32 bits read from TDO are the IDCODE of the last device in the chain, the next 32 bits are the IDCODE of the penultimate device, etc.

Initially, I wanted to show how to do the same thing using a two-wire JTAG, but it turned out that this procedure is a bit too long for an example. Therefore, I decided to simplify it by assuming that there is only one device in the JTAG chain, and the address of its IDCODE register is known. Thus, reading IDCODE will look like this:

1. Write the IDCODE address in the Instruction Register (IR):
 - Go to Shift-IR
 - While in Shift-IR, we push the IDCODE register address into TDI
2. We read data from IDCODE:
 - Go to Shift-DR
 - Being in Shift-DR, we push 32 arbitrary bits into TDI and at the same time we get the contents of IDCODE via TDO

Obviously, being able to perform a simplified procedure, you can easily implement a complete one, because they use the same commands. However, without TDI and TDO, which the 2-wire JTAG does not have, none of the above will work. How is it proposed to do without them? Very simple: Scan-packets containing TMS, TDI and TDO will be transmitted via TMSC. This is how two successive packets look like (nTDI means that an inverted TDI is transmitted):

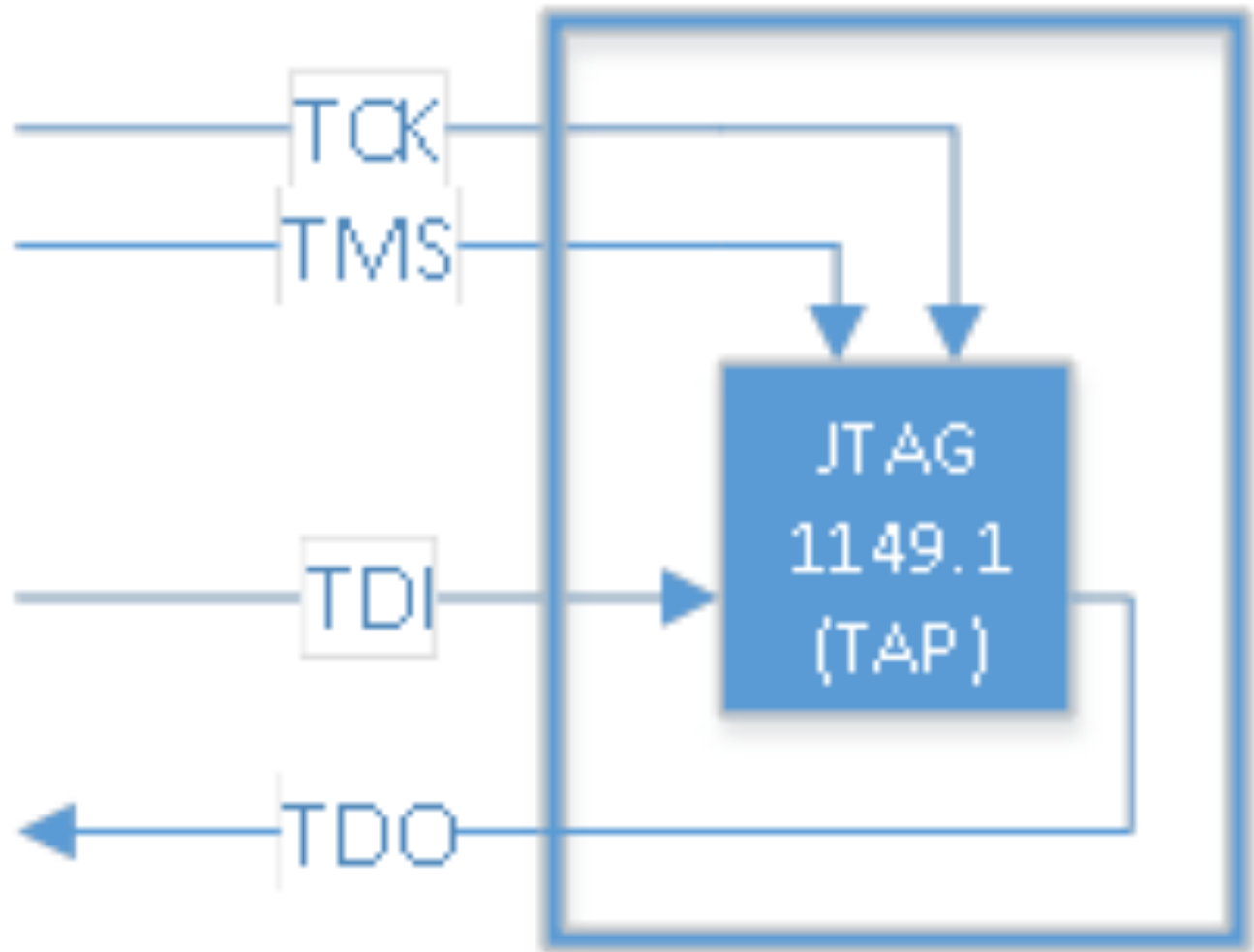


In fact, this is only one of the possible formats described in the standard - it is called Oscan1. There are still JScan, MScan, SScan, and OScan already eight pieces. However, only JScan0-3, MScan, Oscan0 and Oscan1 are mandatory for 2-wire JTAG.

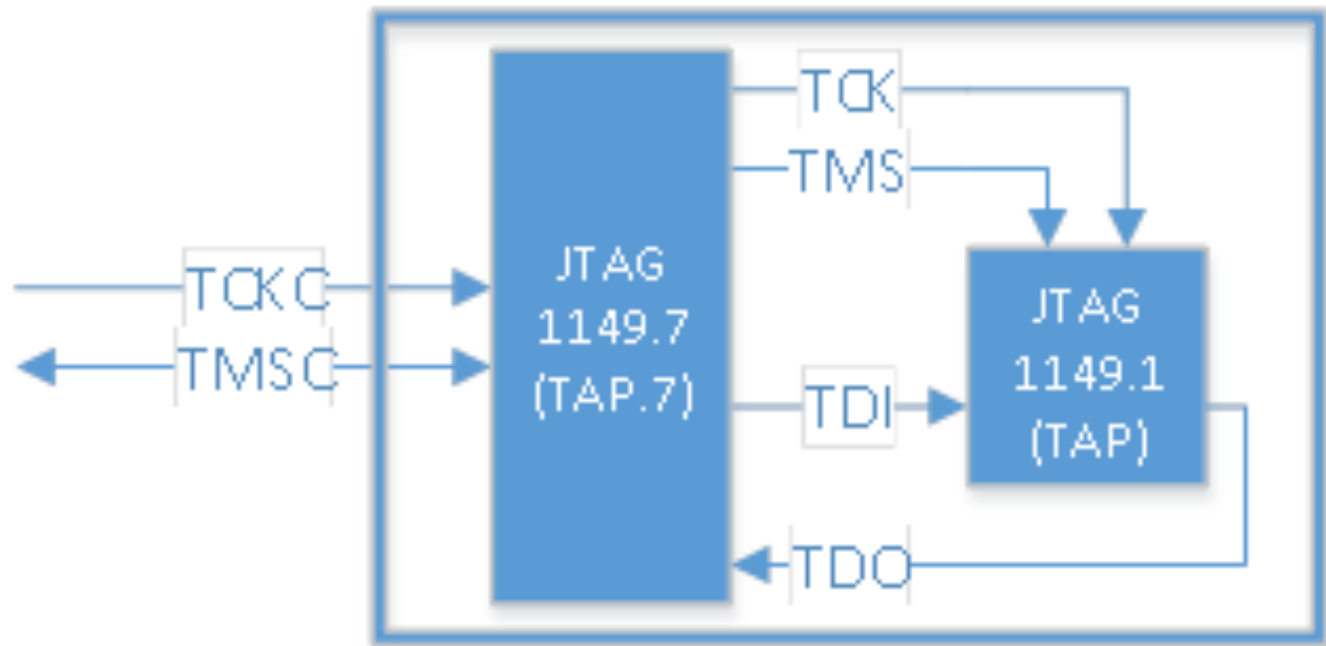
Here it should be noted that the TMSC signal, unlike the usual TMS, is bidirectional, because in each packet TMS and TDI are transmitted in one direction, and TDO - in the other.

However, if you try to just start sending OScan-packages, then you will have an unpleasant surprise. In short, nothing will work. And all because, by default, any device with IEEE 1149.7 support is in compatibility mode with standard 1149.1 (the so-called "standard mode" - standard mode) and does not accept any such packets. Therefore, before sending any OSans, it is necessary to configure such a device to work in the required format and transfer it to "advanced mode" (advanced mode).

Before telling how to do this, a small lyrical digression is required. One of the main goals in the development of IEEE 1149.7 was to ensure compatibility with the existing "iron", which at all times looked like this:

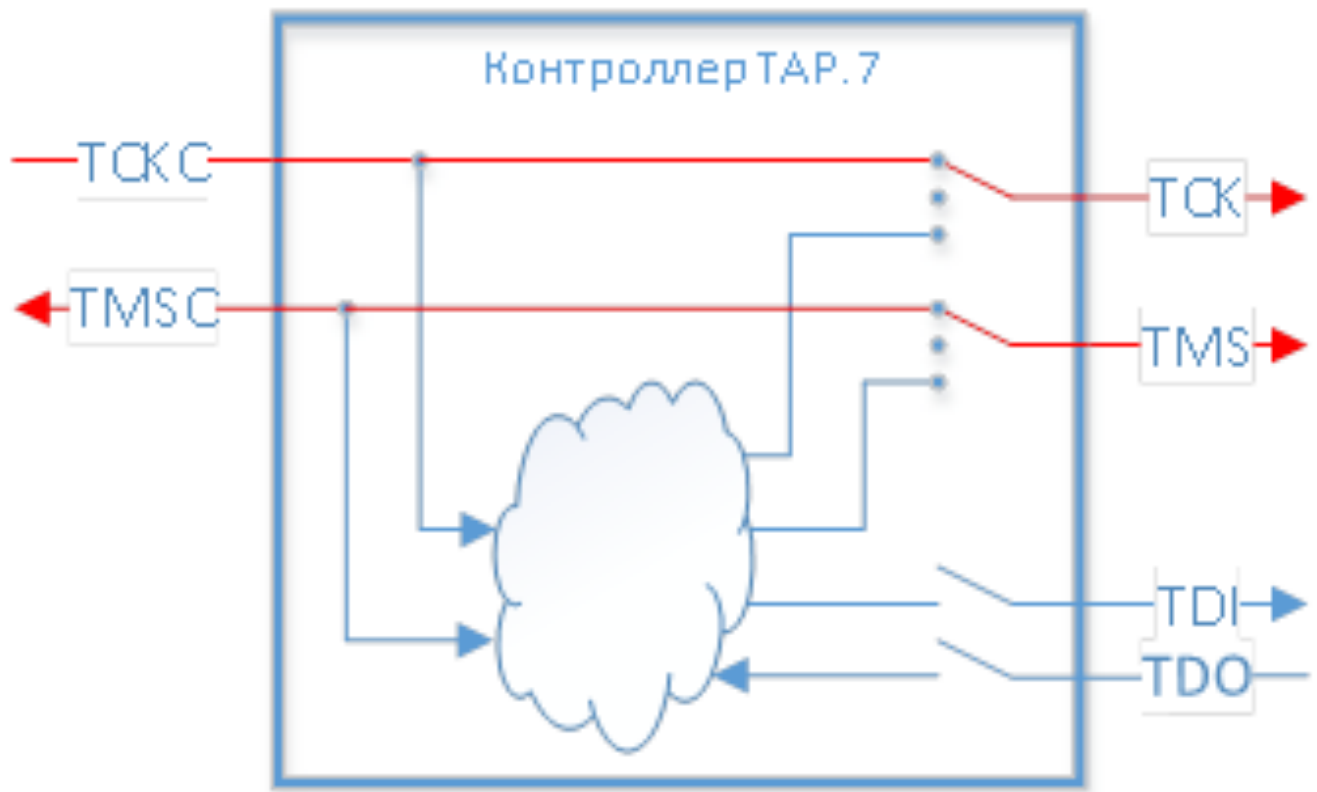


Instead of requiring chipmakers to remake the entire JTAG logic, the standard suggests adding an adapter that converts a two-wire interface to a regular four-wire:

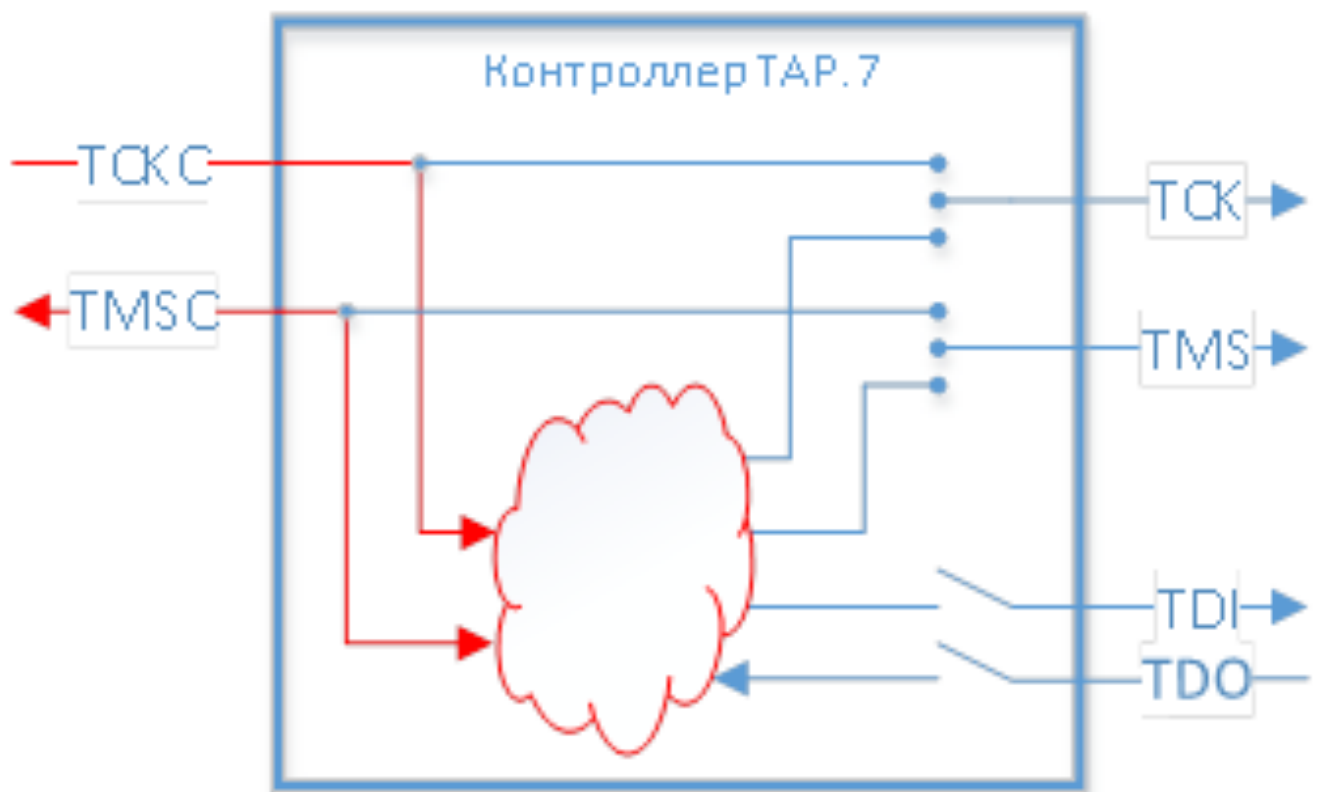


This adapter contains its own Test Access Port (TAP), called TAP.7, so that you can distinguish it from a regular TAP (aka TAP.1). The TAP.7 controller inside is exactly the same machine as TAP.1, but there are some differences, for example, it does not have an Instruction Register, but there are many other registers that TAP.1 does not have.

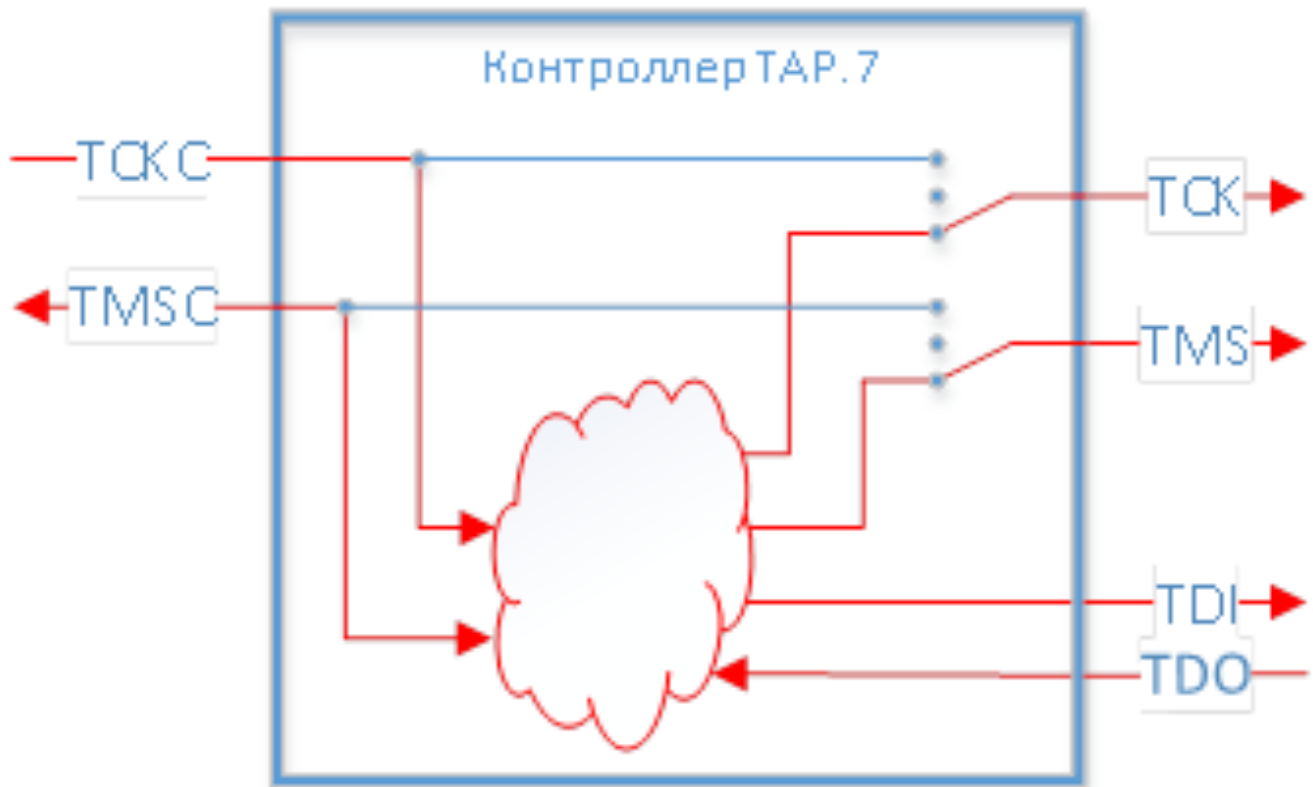
By default, TAP.7 is "transparent", i.e. the TCKC signal is simply directly connected to the TCK, and the TMS is connected to the TMS. Therefore, all signals sent by the TCKC and TMS are transmitted without any changes to the TCK and TMS.



Obviously, when trying to transmit an Oscan1 packet in this mode, the "nTDI" will be perceived by the TAP.1 controller simply as the first bit transmitted via the TMS line, "TMS" as the second bit, etc. That is why before transferring the packets, you need to transfer the TAP.7 controller to the "advanced mode". This is done by writing to one of the TAP.7 internal registers. At the time of configuration, TCK and TMS are disabled (and TDI and TDO were previously disabled):



After the configuration is complete, the TCK, TMS, TDI and TDO are controlled by the controller's internal logic:

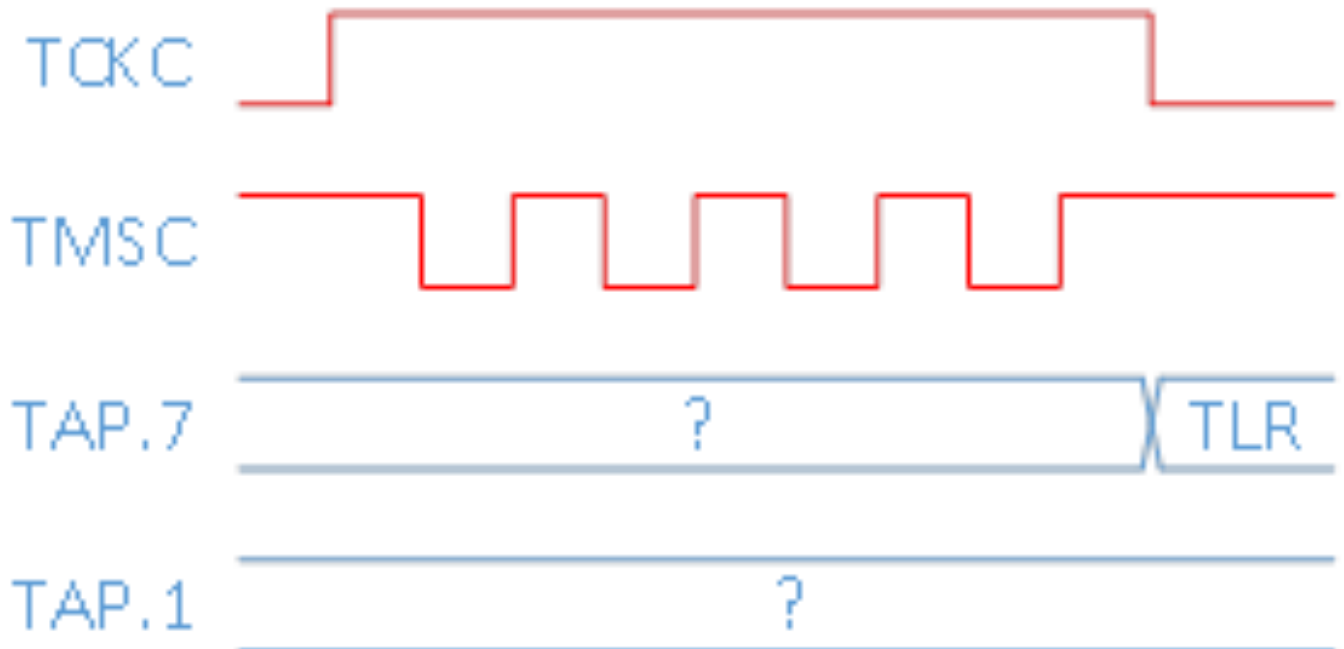


Thus, the process of switching to "advanced mode" and the subsequent reading of IDCODE looks like this:

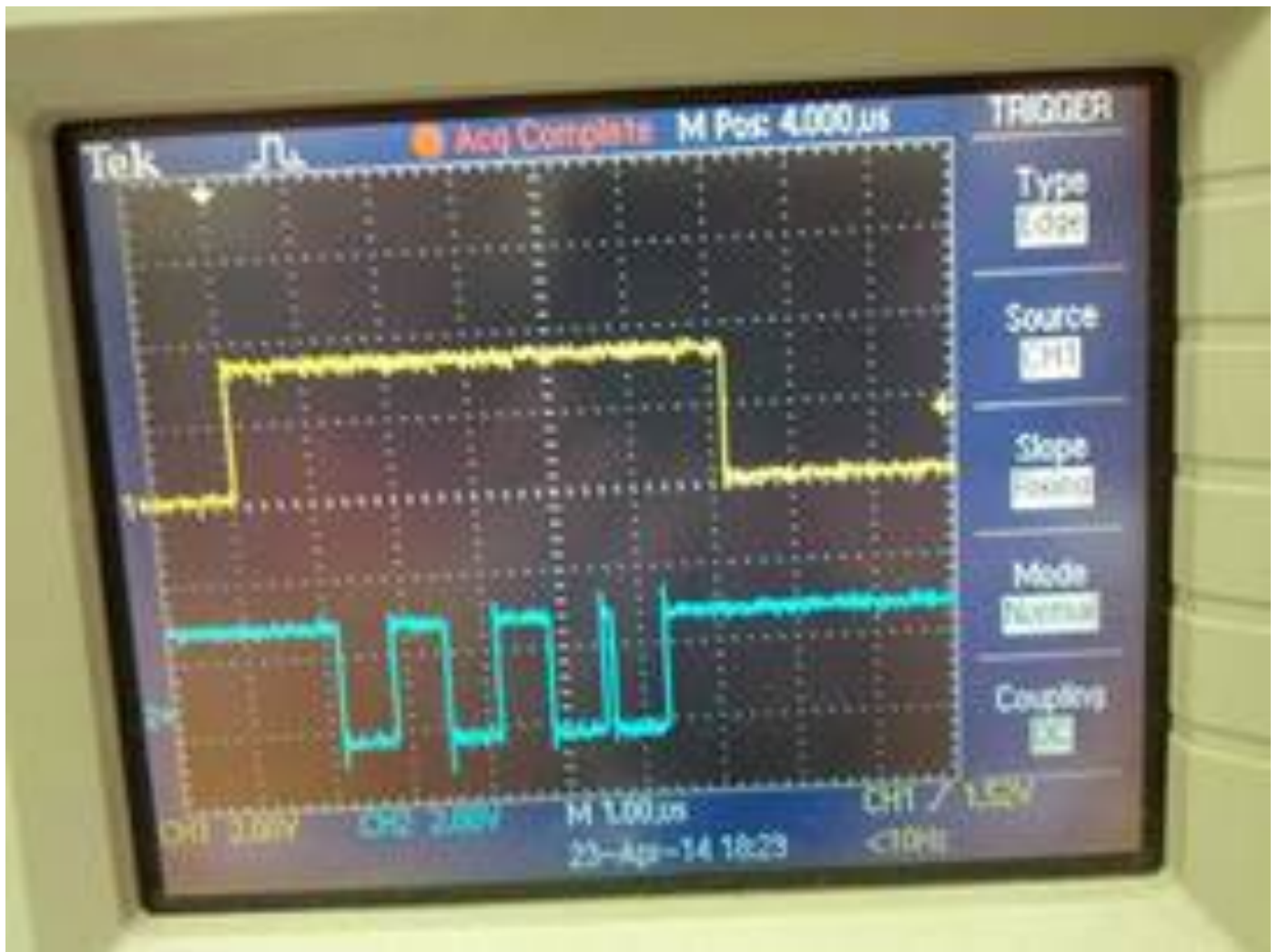
1. Initialize the TAP.7 and TAP.1 controllers
2. Disable TCK and TMS
3. Select the desired transfer format
4. If the format is OScan, MScan or SScan, then go to "advanced mode"
5. Connect TCK, TMS, TDI and TDO
6. Write IDCODE address to the Instruction Register
7. Read IDCODE data

Initializing TAP.7. Escape reset

Since you can never know what state the TAP.7 and TAP.1 machines are in, you need to reset them before starting. To reset the TAP.7 controller, use the Escape Reset, which is one of the types of escape sequences defined by the standard. Escape sequences differ from all other JTAG commands in that the TMS values are not read at the leading edge of the TCK, but asynchronously. The TAP.7 controller counts the number of edges of the TMS signal that came in while the TCKC is unchanged. The number of fronts determines the appearance of the escape sequence. For escape escape it must be more than seven.



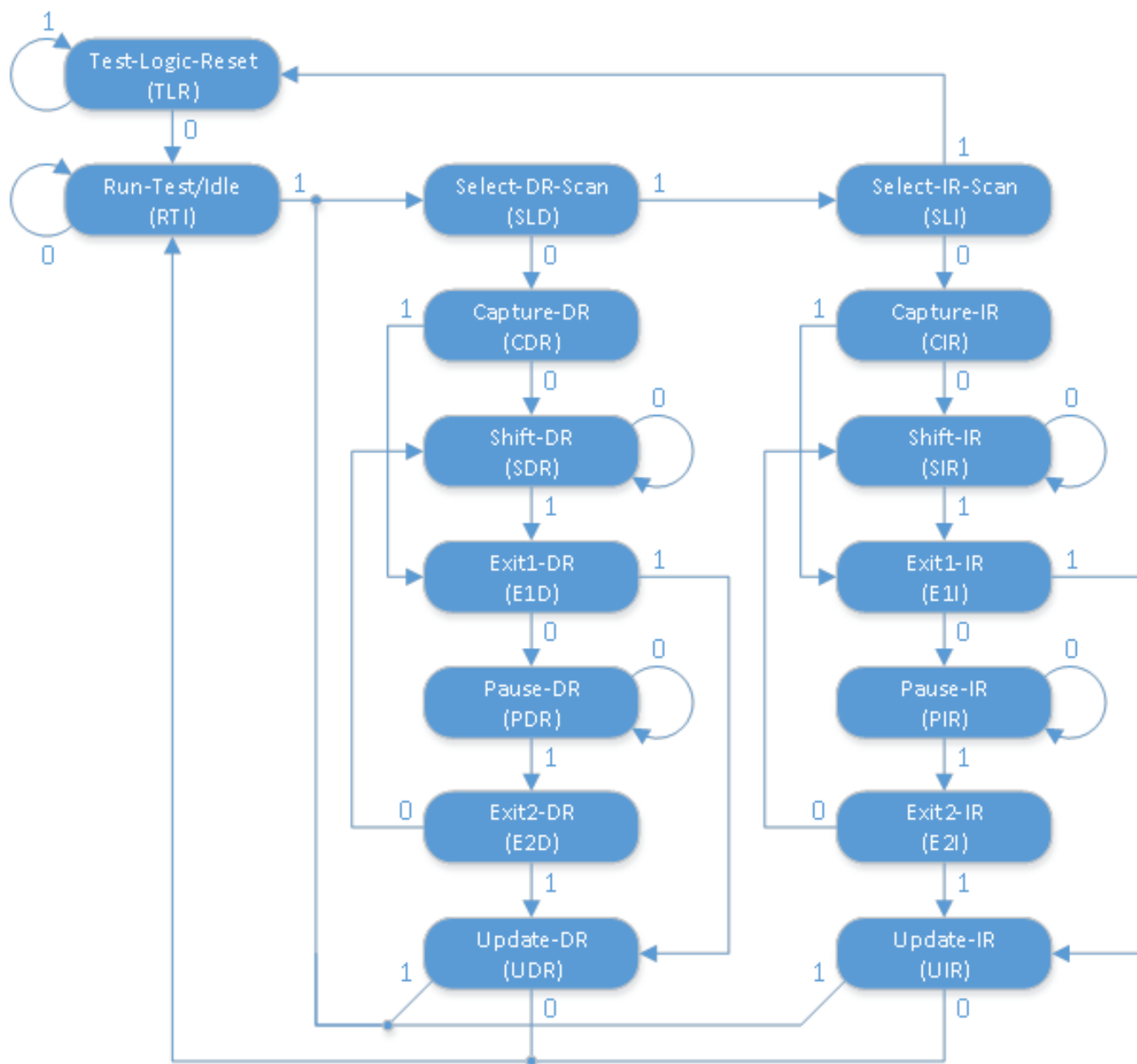
Also, Escape Reset takes TAP.7 to Test Logic Reset. The beauty of escape-sequences is that they have absolutely no effect on TAP.1, which simply ignores them. TAP.1 reads the TMS values only on the leading edge of the TCK, so it doesn't matter to him what happens to TMS the rest of the time. This is how the Escape Reset looks on the oscilloscope screen, which transmits the [Digilent HS2](#) USB-JTAG adapter:



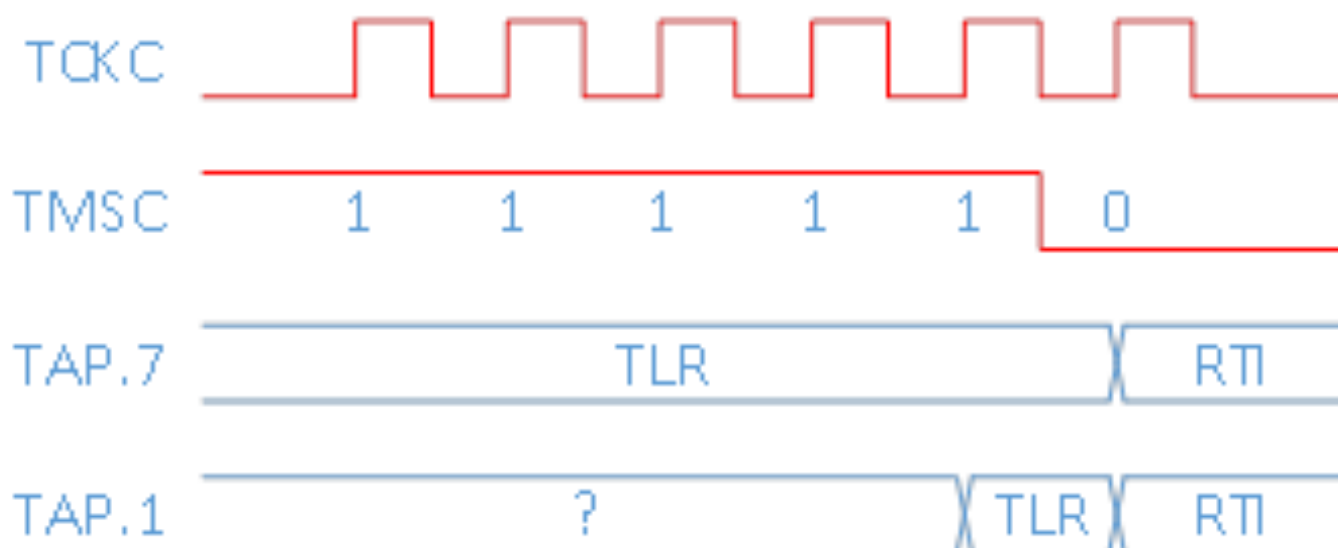
As you can see, all eight fronts are in place, but the distance between the sixth and seventh looks suspicious. I have already complained to Digilent - they promised to fix it.

Initializing TAP1

To reset TAP1 after TAP2, the standard recommends sending five units via TMS0: they are guaranteed to transfer the TAP controller to the Test-Logic-Reset from absolutely any other state (you can check for yourself):



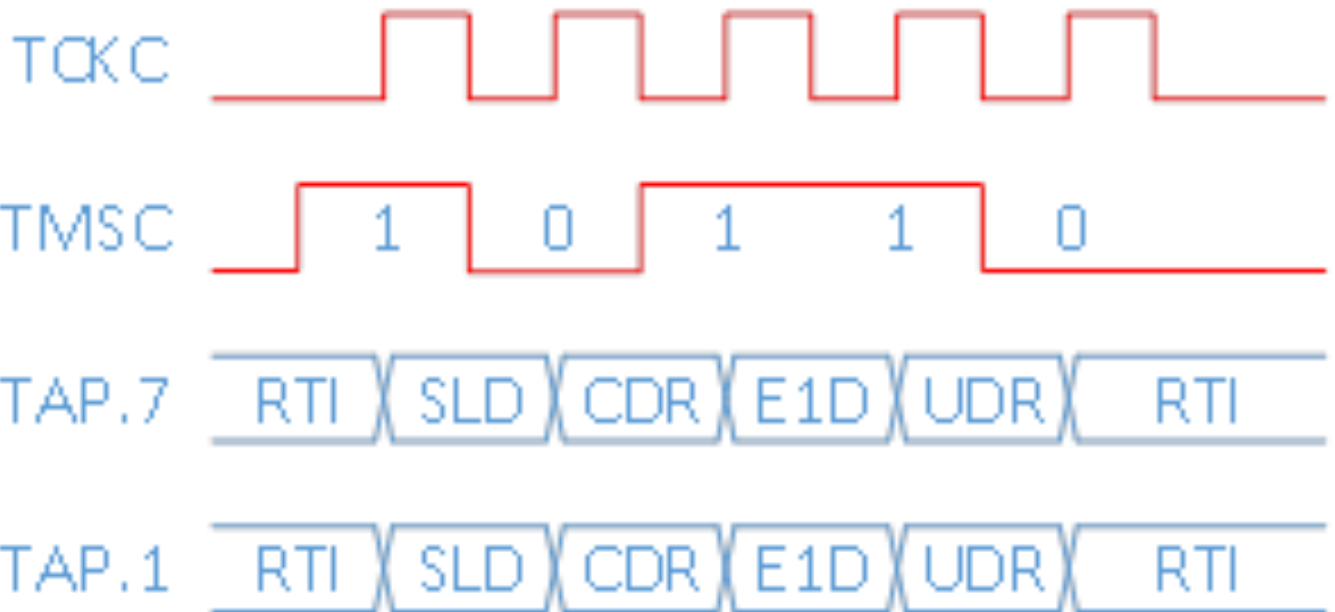
Since we do not plan to return to Test-Logic-Reset anymore, it makes sense to send a zero followed by five units and thereby transfer the machine to the Run-Test / Idle state. Let me remind you that the controller reads the TMS values at the time of the leading edge of the TCK and immediately changes the state of the machine in accordance with these values:



Disable TCK and TMS

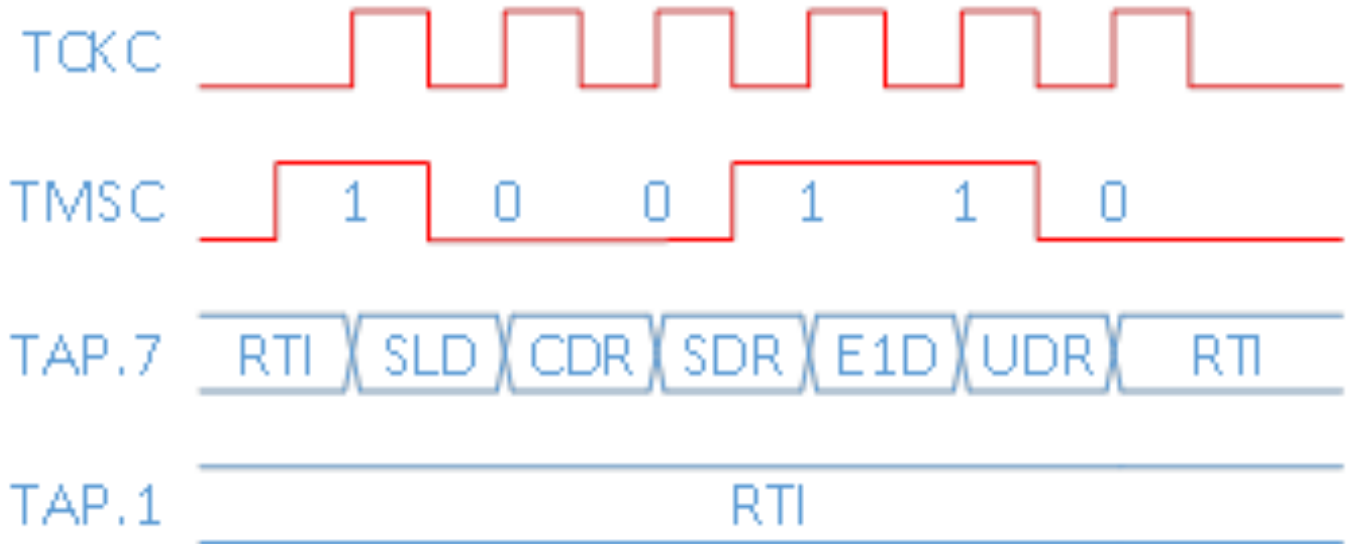
To disable TCM and TMS, you need to transfer the TAP.7 controller to Control Mode 2. In total, the controller supports eight different Control Mode, numbered from zero to seven. We are interested in Control Mode 2, since only in it there is access to the controller's service registers, one of which we need to change the format of the packets on OSCAN1.

Switching between control modes is performed sequentially by performing the so-called Zero-bit DR Shift (ZBS). Each ZBS run increases the Control Mode by one. Unlike escape sequences, in theory ZBS may affect the TAP.1 controller, but in practice this is highly unlikely. From the point of view of the IEEE 1149.1 standard, Zero-bit DR Shift makes absolutely no sense, so the developers hoped that neither hardware nor software developed before the advent of 2-wire JTAG should have used ZBS for any of their needs. On the timeline, ZBS looks like this:



As you can see, ZBS starts and ends in the Run-Test / Idle state, so it's important not to forget to switch to Run-Test / Idle after Escape Reset and Test-Logic-Reset before sending the first ZBS.

When the desired mode is selected, it must be locked (LOCK), after which subsequent ZBS-a will not be able to influence the choice of mode. LOCK is almost the same as ZBS, only with entering Shift-DR:



Notice that the state of TAP.1, unlike TAP.7, does not change. This is because the TCK and TMS are turned off as soon as the Control Mode number becomes equal to two, even if it is not fixed yet.

Thus, to get into Control Mode 2, you need to:

1. Send two zbs
2. Send LOCK

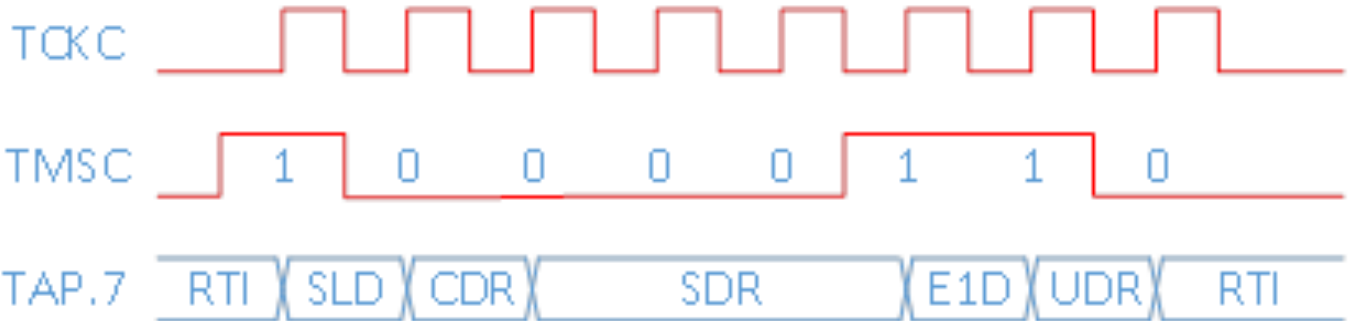
Format selection

Now that we are in Control Mode 2, we can write to the service registers. The standard describes a set of registers, the entry in which is made by special teams. The teams are divided into two groups - two-part (two-part command) and three-part (three-part command), which I will not consider.

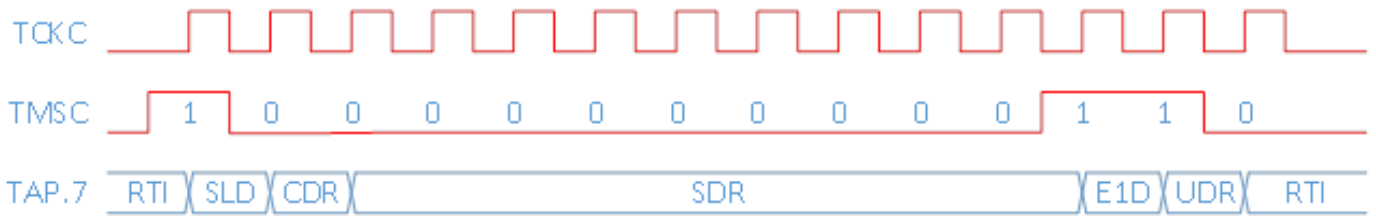
In the case of two-section commands, the command number is transmitted in the first section, and the data is transmitted in the second section. Both sections are transmitted in the same way: from the Run-Test / Idle state you need to go to Shift-DR, and then return to Run-Test / Idle. The value transmitted is equal to the number of clock cycles held in Shift-DR.

We are interested in the command number three - STFMT (Store Scan Format), which writes the number of the desired format (for OSCan1 it is equal to nine), transmitted in the second section, to the register SCNFMT (Scan Format).

Thus, in the first section we need to transfer the top three, and in the second - the nine. First, we write down the command number, for which we are waiting for three cycles in the Shift-DR state (the TAP.1 state is not shown, since it does not change):



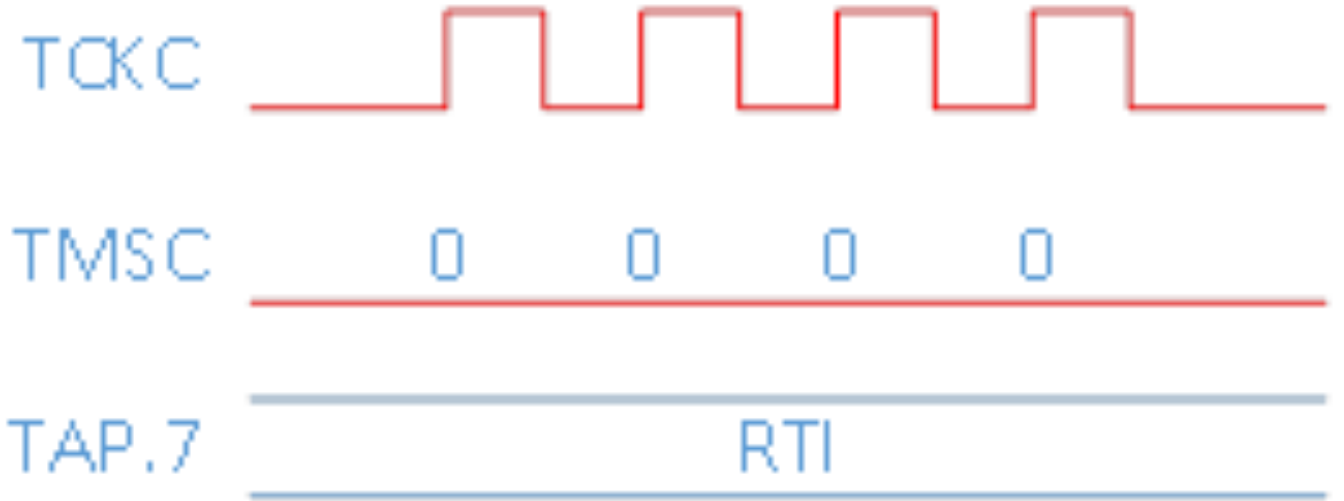
Then we write the data (nine clock cycles in the Shift-DR state):



Writing to the SCNFMT register itself does not change the format. In order for the controller to switch the format, you need to send a special check packet (check packet).

Check packet

The test package checks the correctness of the settings, and if everything is in order, the controller switches the format and, if necessary, switches to the "advanced mode". Like everything else in IEEE 1149.7, the test package has a bunch of different options. You will have to take my word for it that we need a package with an END directive (and do not ask what this means), which looks like this:



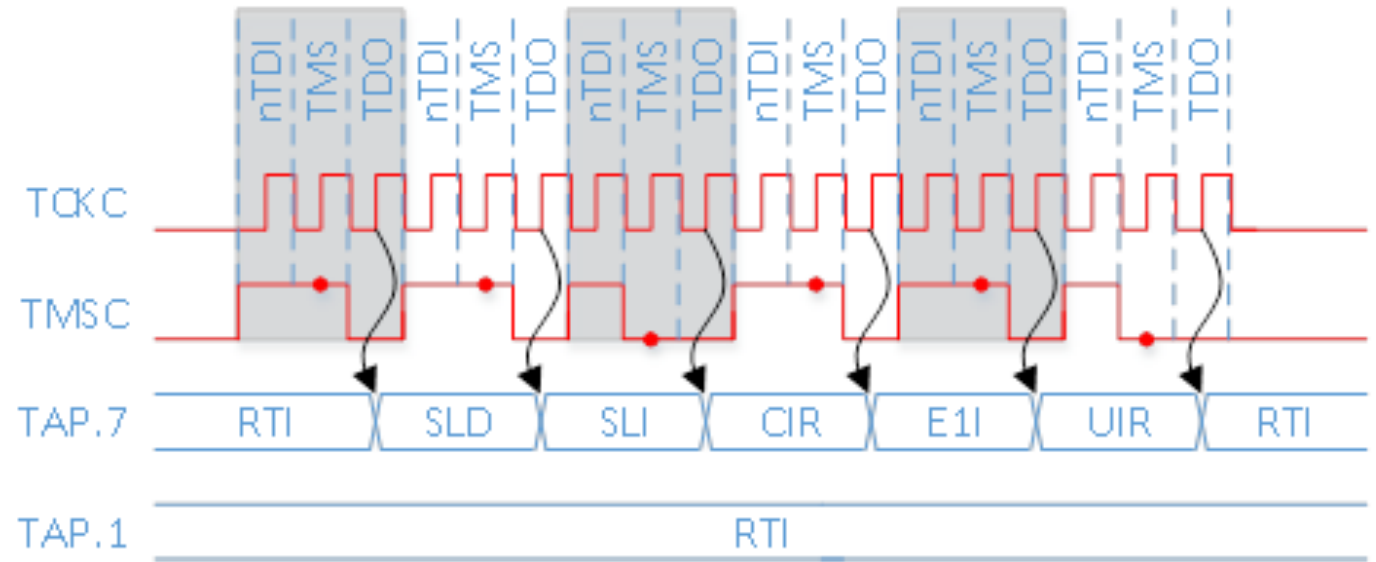
Hoorary, now we are in "advanced mode" and we can finally use OScan!! To return to the "standard mode", just go to Test-Logic-Reset.

Connecting TCK, TMS, TDI and TDO

So, we are in "advanced mode", but still in Control Mode 2, which means that TAP.1 is still disabled, and we cannot read IDCODE from it. The standard describes four ways to exit Control Mode:

1. Go to Test Logic Reset
2. Go to Select-IR-Scan
3. Write one to ECL (Exit Control Level) using a two-section STMC command
4. To do something unknown, associated with the synchronization of TAP.7-controllers

In Test-Logic-Reset we can not, and then the controller will return to the "standard mode". Two-team command to send a long time. But to go to Select-IR-Scan and return to Run-Test / Idle, without going through Test-Logic-Reset, is quite simple - you just need to send the sequence 011011 by TMS (it is not necessary to enter Shift-IR). Remember that in JTAG, the least significant bit is always transmitted first:



Red dots mark the places where the controller reads the TMS bits from the TMSC input. Zeros are transmitted via TDI (i.e., in all packets, the nTDI bits are equal to one), although they are still ignored, since we do not go into the Shift-IR state.

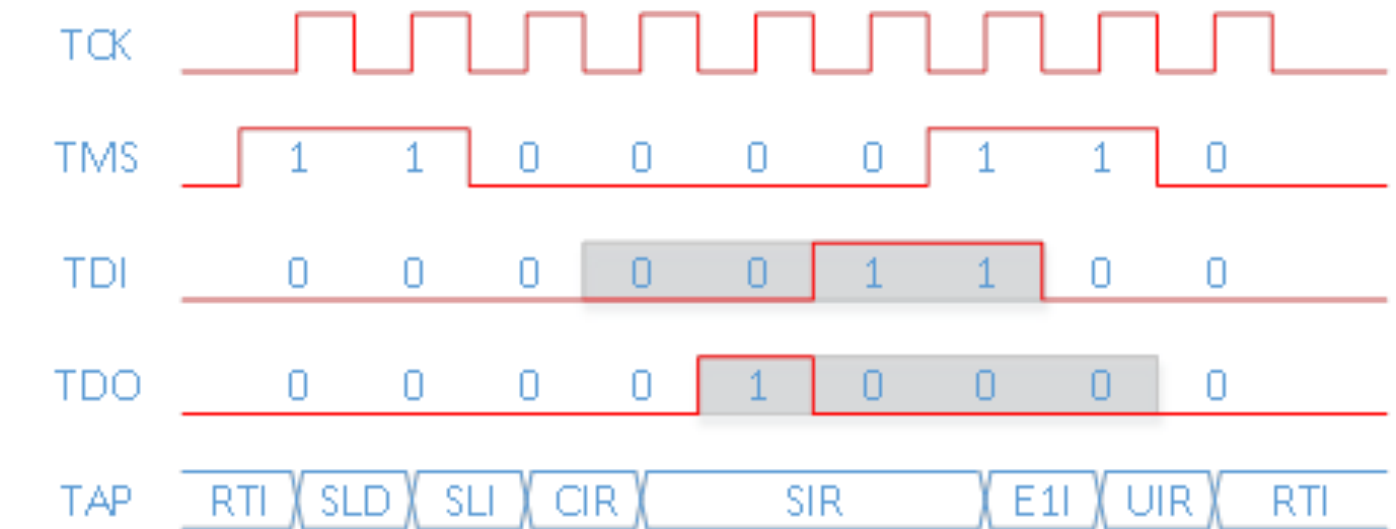
By the way, unlike "standard mode", the state of TAP.7 does not change immediately after receiving a new TMS bit, but on the third (last) leading edge of TCKC in the OScan1 packet (in the figure these points are shown by black arrows) - this is done in order for the controller to insert the TDO bit into OScan1-packet and send it to the host (in this particular case, the controller returns zeroes via TDO).

Reading IDCODE

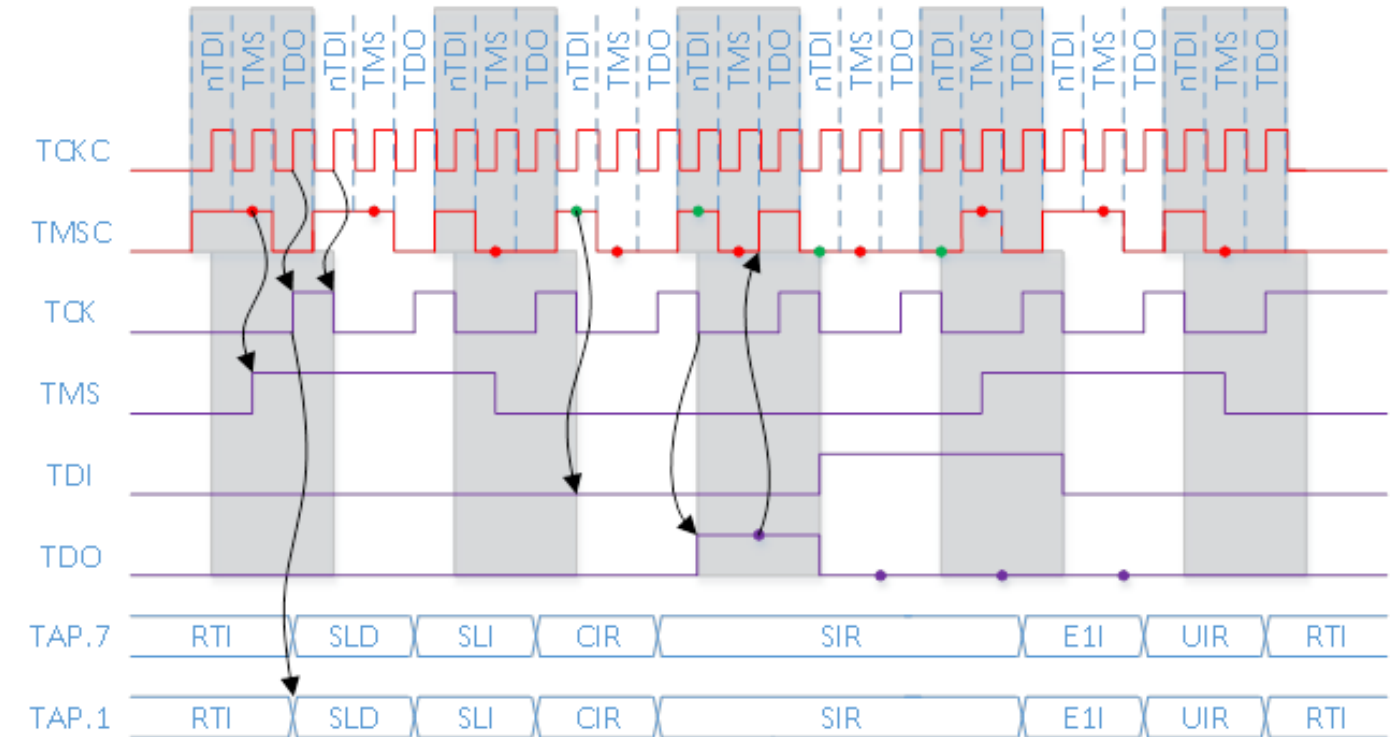
Now that TAP.1 is again connected to TAP.7, we can finally read the contents of the IDCODE register.

The address of the IDCODE register is not described in the standard, therefore in all controllers it is different (in my case it is four-bit and is equal to 0xC). To make life easier for users, the Instruction Register is initialized with an IDCODE address in the Test-Logic-Reset state. However, in the Capture-IR state, the contents of the Instruction Register are overwritten with the number 0x1 (regardless of the length of the Instruction Register, its low-order bit is set to one, and all the others to zero) so that an open circuit can be detected during testing. Since we entered Capture-IR in the previous step, the IDCODE address is already overwritten, so you first need to overwrite it.

To write the register address, you must enter the Shift-IR state and push the address (0xC) along the TDI in the low-order bit. In this case, the value that was in the Instruction Register before, i.e. 0x1 (again, the low bit forward). In the case of a conventional four-wire IEEE 1149.1 JTAG, it would look like this:



And this is how it looks when using OScan1 packages:



Red dots mark the places where the controller reads the TMS bits, and the green dots show the nTDI bits from the TMSC input. Purple dots mark the places where the controller reads the TDO bits before inserting them into OScan1 packets. The arrows show some dependencies, for example, the TMS and TDI signals vary along the corresponding edges of the TCKC, and the TDO signal changes along the falling edge of the TCK.

Now it only remains to go to the Shift-DR state, push 32 arbitrary bits on TDI and read 32 bits from TDO, which will contain IDCODE. With your permission, I will not provide a temporary IDCODE reading diagram.

Conclusion

I have described about 5% of the IEEE 1149.7 capability. Nevertheless, I hope that you have some understanding of the principles underlying it.

All diagrams in this article are tested on real hardware using the Digilent HS2 USB-JTAG adapter, a commercial 2-wire JTAG controller, to the source codes of which I have access, a debugging board with FPGA and a Xilinx ChipScope logic analyzer.

The program for generating sequences was written using the API for HS2 based on an example, which can be downloaded along with the [Digilent Adept SDK](#) and worked fine under both Linux and Windows.

Source: <https://habr.com/ru/post/237885/>

More articles:

- [SALT: smart beacon protects wallet and phone](#)
- [BaitClub/Moscow 2014: interview with Kirill Timofeyev](#)
- [letBrains distributes free products to students](#)
- [Guide to creating startups. part 4: the most important thing for a startup](#)
- [Welcome to the 2nd Moscow Django Meetup](#)
- [Backblaze updated HDD reliability statistics](#)
- [ORM or how to forget about database design](#)
- [WGDG Competition: Latest Recommendations and Jury Composition](#)
- [The main types of search queries that online store users use: is your website ready for them? \(Part 1\)](#)
- [Using SmtipClient to send mail via Yandex SMTP server with SSL](#)

All Articles

This site may contain Advertising and Affiliate Program links. As an Amazon Associate I earn from qualifying purchases.